



Vertex Enterprise Services: Containerized Multi-Service Web Platform

Deployment and Recreation Guide

Ernest Barron

6/28/2026

Table of Contents

Table of Contents.....	2
Introduction.....	3
Project Overview.....	3
System Requirements.....	4
Setting Up Ubuntu Instance.....	4
Step 1: Create the Ubuntu Instance.....	4
Step 2: Network Configuration.....	5
Step 3: Installing Docker.....	5
Step 4: Configure Docker.....	7
Step 5: Install Docker Compose.....	7
Development Workflow and Source Control.....	8
Installing Git.....	8
Version Control Workflow.....	8
Updating the Development Environment.....	9
Creating the Base Container.....	9
Prepare a Static Website.....	9
Dockerfile.....	9
Building the Base Container Image.....	10
Run the Container.....	11
Creating the Docker Compose Environment.....	11
Creating Multiple Identical Services with Docker Compose.....	12
Docker Compose with Multiple Services.....	14
Conclusion.....	16
Appendix A - Source Code Repositories.....	17
BaseContainer Repository.....	17
Project-2 Repository.....	17
Using the Repositories.....	17

Introduction

The purpose of this document is to describe the complete process for recreating the Vertex Enterprise Services containerized environment. Following the procedure in this guide will allow another user to provision an Ubuntu server, install Docker and Docker compose, build the required container images, and deploy the application exactly as originally developed.

This guide assumes no prior project configuration exists and begins with a clean Ubuntu instance.

Project Overview

Vertex Enterprise Services is composed of five independent web services representing different business units of the Vertex Virtual Workspaces organization.

Service	Purpose	Ports
Headquarters	Corporate Homepage	8081
Cloud Services	Cloud Infrastructure	8082
Security	Cybersecurity Solutions	8083
Customer Success	Customer Support and Success	8084
Research & Innovation	Research Initiatives	8085

Each service is deployed inside its own Docker container.

System Requirements

This project was developed using the following environment:

Component	Version
AWS Lightsail	Ubuntu 24.04 LTS
Docker Engine	Latest Stable
Docker Compose	v2
Nginx	stable-alpine

Setting Up Ubuntu Instance

Step 1: Create the Ubuntu Instance

Create a new AWS Lightsail Ubuntu instance with the recommended configuration:

- Linux operating system
- Ubuntu 24.04 LTS
- General purpose instance plan
- Dual-stack network type
- 2GB memory
- 2vCPUs Processing
- 60 GB SSD
- 3 TB Transfer

Once created, you may connect to your instance via Lightsails browser-based SSH client.

Connect to your instance [Info](#)

You can connect using your browser, or your own compatible SSH client.

Use your browser [Info](#)

Connect using our browser-based SSH client.

 **Connect using SSH**

Step 2: Network Configuration

To ensure that we are able to access our services, we must add additional inbound ports for HTTP and ICMP protocols.

To do this, proceed to the Networking tab in the Lightsail console of your instance and add the following rules for IPv4 and IPv6:

Specify a port and protocol to open. Specify a port range using a dash, such as 0 - 65535.

Application	Protocol	Source IP address
All ICMP	ICMP	Any IPv4 address

Specify a port and protocol to open. Specify a port range using a dash, such as 0 - 65535.

Application	Protocol	Port or range	Source IP address
Custom	TCP	8080 - 8085	Any IPv4 address

Step 3: Installing Docker

1. Adding Docker's official GPG key to verify packages downloaded from the Docker repository:

```
sudo apt-get update

sudo apt-get install ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL http://download.docker.com/linux/ubuntu/gpg
-o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc
```

2. The Docker repository is then added to Ubuntu's package manager before installing Docker Engine:

```
echo \

"deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
```

```
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
sudo apt-get update
```

3. Install the Docker engine and test it:

```
sudo apt-get install docker-ce docker-ce-cli containerd.io  
docker-buildx-plugin docker-compose-plugin  
  
sudo docker run hello-world
```

Expected output:

```
ubuntu@ip-172-26-5-70:~$ sudo docker run hello-world  
Hello from Docker!  
This message shows that your installation appears to be working correctly.
```

Step 4: Configure Docker

By default the Docker daemon can only be accessed by the root user or users who belong to the docker group. Without additional configuration, every Docker command would need to be prefixed with `sudo`.

To allow Docker commands to be executed without elevated privileges, the current user is added to the docker group. This improves usability during development while maintaining Docker's standard permission model.

The following command configure Docker permissions:

```
sudo groupadd docker

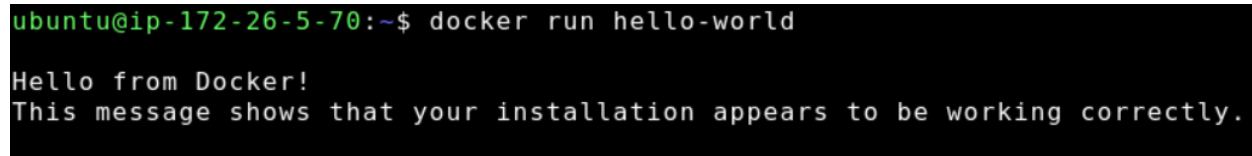
sudo gpasswd -a $USER docker

newgrp docker
```

After configuring permissions, verify that Docker daemon is accessible using the current user account:

```
docker run hello-world
```

Expected output:

A terminal window with a black background. The prompt is 'ubuntu@ip-172-26-5-70:~\$'. The command 'docker run hello-world' has been entered. The output is 'Hello from Docker!' followed by 'This message shows that your installation appears to be working correctly.'

```
ubuntu@ip-172-26-5-70:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

If the container runs successfully, the Docker daemon is accessible using the current user account.

Step 5: Install Docker Compose

Docker Compose is used to define and manage multiple containers as a single application. Rather than creating and starting each container individually. Docker Compose reads the `docker-compose.yml` and automatically builds the required images, creates a shared network, and starts all services in the correct order.

For this project, Docker compose is responsible for orchestrating the five Vertex service containers.

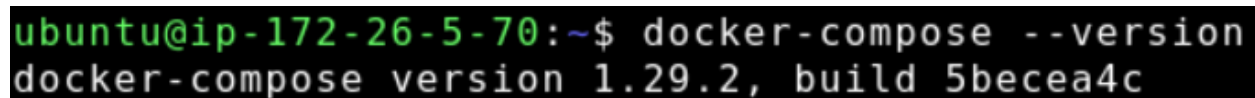
Execute the following commands to install and verify:

```
sudo curl -L
"https://github.com/docker/compose/releases/download/1.29.2/docke
r-compose-$(uname -s)-$(uname -m)" -o
/usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose

docker-compose --version
```

Expected output:



```
ubuntu@ip-172-26-5-70:~$ docker-compose --version
docker-compose version 1.29.2, build 5becea4c
```

Development Workflow and Source Control

Development of the Vertex Enterprise Service project was performed using Visual Studio Code on a native Windows workstation. All project files, including HTML, CSS, Dockerfiles, and Docker Compose configuration, were created and maintained locally. This approach provided a full-featured development environment with source control integration, syntax highlighting, and file management capabilities.

To facilitate version control and simplify development to the Ubuntu instance, the project was hosted in a GitHub repository. The Ubuntu server acted as the deployment environment rather than the primary development environment.

Installing Git

In order to achieve the desired workflow, we must ensure that Git is properly installed in the Ubuntu environment. To do this, run the following code:

```
sudo apt update

sudo apt install git
```

Version Control Workflow

After making changes locally, updates were committed and pushed to the GitHub repository using Git. This ensured that the repository always contained the most recent version of the project.

The repository was cloned onto the Ubuntu instance using:

```
git clone https://github.com/<username>/<repository>.git
```

Updating the Development Environment

Whenever changes were made on the local development machine and pushed to GitHub, the Ubuntu instance was synchronized by navigating to the project directory and pulling the latest changes:

```
git pull
```

After pulling updated files, any modified Docker images were rebuilt before restarting the service. The steps for restarting the Docker images will be covered in a later section.

Creating the Base Container

The first stage of the development process is creating the base Docker container that will serve as the foundation for each Vertex Enterprise service. Docker packages the website files and the Nginx web server into a portable image, allowing the application to run consistently regardless of the host operating system.

For this project, every service is built from the same base image. Although each service contains different HTML, CSS, and image assets, they all use the same underlying Docker configuration.

Prepare a Static Website

On your local machine, prepare a static website that you will use as the blueprint for your Docker image and save it to a GitHub repository.

Your static website should follow this common structure:

```
| - index.html
| - styles
    | - styles.css
| - img
    | - <your-img>.png
```

Dockerfile

Create a file named `Dockerfile` in the root directory of the service. In the file write the following four lines of code:

```
FROM nginx:stable-alpine
COPY . /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Commit these changes to your GitHub repository. In your Ubuntu instance, clone the repository to create a working copy of your static website and Dockerfile.

Building the Base Container Image

After creating the Dockerfile, the container can be built using Docker:

```
docker build -t my-nginx-container .
```

And verify that the image has been created:

```
docker image ls
```

Expected output:

```
ubuntu@ip-172-26-5-70:~/BaseContainer$ docker build -t my-nginx-container .
[+] Building 0.5s (7/7) FINISHED          docker:default
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 135B                      0.0s
=> [internal] load metadata for docker.io/library/nginx:s 0.2s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                            0.0s
=> [internal] load build context                        0.0s
=> => transferring context: 2.20kB                       0.0s
=> [1/2] FROM docker.io/library/nginx:stable-alpine@sha25 0.0s
=> => resolve docker.io/library/nginx:stable-alpine@sha25 0.0s
=> CACHED [2/2] COPY . /usr/share/nginx/html           0.0s
=> exporting to image                                  0.1s
=> => exporting layers                                    0.0s
=> => exporting manifest sha256:8f76cbd56e8e19f3c8ba94391 0.0s
=> => exporting config sha256:7c0dff4a91a2d7fceb4970975a 0.0s
=> => exporting attestation manifest sha256:e1569191e4f04 0.0s
=> => exporting manifest list sha256:d0b6c07c6de172795a68 0.0s
=> => naming to docker.io/library/my-nginx-container:late 0.0s
=> => unpacking to docker.io/library/my-nginx-container:l 0.0s
ubuntu@ip-172-26-5-70:~/BaseContainer$ docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
hello-world:latest	96498ffd522e	25.9kB	9.49kB	U
my-nginx-container:latest	d0b6c07c6de1	93.6MB	26.5MB	

Run the Container

Before completing multiple services, it is important to verify that the base container functions correctly.

Run the container using:

```
docker run -p 8080:80 my-nginx-container
```

To verify that the container is running, open a web browser on your local machine and navigate to `http://<SERVER_IP>:8080`.

If the container has been configured correctly, the homepage should be displayed.



Successful completion of this step confirms that:

- Docker is functioning correctly.
- Nginx is serving the website.
- Network port mapping is working properly.
- The website assets have been copied successfully into the container

Creating the Docker Compose Environment

After validating that a single container runs correctly, the next step is to define a multi-container architecture using Docker Compose. This allows all Vertex Enterprise Services to be deployed, networked, and managed as a single application. Each service will run as an independent container but share a common Docker network and base configuration.

Creating Multiple Identical Services with Docker Compose

Before creating the multi-service application, Docker Compose should be tested by launching five identical copies of the same container. This will verify that Docker Compose successfully builds, creates, and manages multiple services from a single configuration file.

The `docker-compose.yml` should look similar to the below implementation:

```
version: "3.9"

services:

  web1:
    image: my-nginx-container
    container_name: vertex-web1
    ports:
      - "8081:80"

  web2:
    image: my-nginx-container
    container_name: vertex-web2
    ports:
      - "8082:80"

  web3:
    image: my-nginx-container
    container_name: vertex-web3
    ports:
      - "8083:80"

  web4:
    image: my-nginx-container
    container_name: vertex-web4
    ports:
      - "8084:80"

  web5:
    image: my-nginx-container
    container_name: vertex-web5
    ports:
      - "8085:80"
```

Build and launch the service and verify that all five containers are running:

```
docker-compose up --build -d
```

```
docker ps
```

Expected output:

```
ubuntu@ip-172-26-5-70:~/BaseContainer$ docker-compose up --build -d
Creating network "basecontainer_default" with the default driver
Creating vertex-web4 ... done
Creating vertex-web1 ... done
Creating vertex-web2 ... done
Creating vertex-web5 ... done
Creating vertex-web3 ... done
ubuntu@ip-172-26-5-70:~/BaseContainer$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
AMES          4583042aa09a   my-nginx-container      "/docker-entrypoint..." 6 seconds ago Up 5 seconds  0.0.0.0:8081->80/tcp
ertex-web1    60a65c9d6c8c   my-nginx-container      "/docker-entrypoint..." 6 seconds ago Up 5 seconds  0.0.0.0:8085->80/tcp
ertex-web5    ce31c010bbb9   my-nginx-container      "/docker-entrypoint..." 6 seconds ago Up 5 seconds  0.0.0.0:8082->80/tcp
ertex-web2    44a861dec99f   my-nginx-container      "/docker-entrypoint..." 6 seconds ago Up 5 seconds  0.0.0.0:8083->80/tcp
ertex-web3    b564fba34b86   my-nginx-container      "/docker-entrypoint..." 6 seconds ago Up 5 seconds  0.0.0.0:8084->80/tcp
ertex-web4
ubuntu@ip-172-26-5-70:~/BaseContainer$
```

Finally, access each instance by visiting:

```
http://<SERVER_IP>:8081
http://<SERVER_IP>:8082
http://<SERVER_IP>:8083
http://<SERVER_IP>:8084
http://<SERVER_IP>:8085
```

To stop running the containers:

```
docker-compose down
```

Although each container displays the same webpage, this exercise demonstrates that Docker Compose can manage multiple instances of a single service from one configuration file.

Docker Compose with Multiple Services

After verifying that Docker Compose could successfully manage multiple identical containers, the environment was expanded to support multiple unique services. Rather than deploying five copies of the same website, each service was modified to represent different departments within the Vertex Enterprise Services organization while continuing to use the same Docker and Nginx foundation.

Each service contains its own directory consisting of an `index.html` file, a stylesheet, and image assets. Although the website content differs between departments, every service is built using the same base container configuration.

The project directory is organized as follows:

```
root/
|- docker-compose.yml
|- hq/
|- cloud/
|- security/
|- customer/
|- research/
```

The `docker-compose.yml` file is then updated so that each service builds from its corresponding directory and exposes a unique host port:

```
version: "3.8"

services:

  hq:
    build: ./hq
    container_name: hq
    ports:
      - "8081:80"

  cloud:
    build: ./cloud
    container_name: cloud
    ports:
      - "8082:80"

  security:
    build: ./security
    container_name: security
```

```

    ports:
      - "8083:80"

customer:
  build: ./customer
  container_name: customer
  ports:
    - "8084:80"

research:
  build: ./research
  container_name: research
  ports:
    - "8085:80"

```

Once the configuration file has been updated, pull the updated resources via Git, rebuild the images, and start all service:

```

git pull

docker compose up --build -d

```

Verify that each container is running:

```

docker ps

```

Expected output should display five running containers, each mapped to a different host port:

IMAGE	COMMAND	CREATED	STATUS	PORTS
project-2_security	"/docker-entrypoint..."	6 minutes ago	Up 6 minutes	0.0.0.0:8083->80/tcp,
project-2_cloud	"/docker-entrypoint..."	6 minutes ago	Up 6 minutes	0.0.0.0:8082->80/tcp,
project-2_research	"/docker-entrypoint..."	6 minutes ago	Up 6 minutes	0.0.0.0:8085->80/tcp,
project-2_customer	"/docker-entrypoint..."	6 minutes ago	Up 6 minutes	0.0.0.0:8084->80/tcp,
project-2_hq	"/docker-entrypoint..."	6 minutes ago	Up 6 minutes	0.0.0.0:8081->80/tcp,

Finally, verify each website by navigating to the following address in a local web browser:

```

http://<SERVER_IP>:8081
http://<SERVER_IP>:8082
http://<SERVER_IP>:8083
http://<SERVER_IP>:8084
http://<SERVER_IP>:8085

```

Each page should display unique branding, text, and imagery while sharing the same underlying Docker architecture.

Conclusion

The Vertex Enterprise Services platform successfully demonstrates a fully containerized multi-service web architecture using Docker and Docker Compose.

Through this deployment process, the following objectives were achieved:

- A clean Ubuntu-based cloud environment was provisioned on AWS Lightsail.
- Docker Engine and Docker Compose were installed and configured.
- A reusable base Nginx container was defined using Docker Compose.
- Five independent business services were deployed as isolated containers.
- Source control integration was implemented using GitHub for development and deployment synchronization.
- Horizontal scaling was demonstrated through container duplication.

This architecture provides a foundation for scalable, modular web services and reflects common real-world DevOps deployment workflows used in containerized enterprise environments.

Appendix A - Source Code Repositories

The complete source code used throughout this development guide is available in two separate GitHub repositories. These repositories are provided as reference material and allow readers to compare their implementation with the original project files or deploy the application directly.

BaseContainer Repository

The `BaseContainer` repository contains the introductory implementation used during the early stages of development. This repository includes:

- Initial static website
- Supporting HTML, CSS, and image assets
- Dockerfile
- Initial `docker-compose.yml`

Repository:

<https://github.com/ebarron03/BaseContainer>

Project-2 Repository

The `Project-2` repository contains the completed Vertex Enterprise Services project described throughout this guide. This repository includes:

- Five unique website services
- Department-specific HTML, CSS, and image assets
- Individual Dockerfiles
- Final Docker Compose configuration
- Final containerized multi-service implementation

Repository:

<https://github.com/ebarron03/Project-2>

Using the Repositories

Readers who wish to recreate the project using the original source files may clone either repository using Git:

```
git clone https://github.com/ebarron03/BaseContainer.git
```

```
git clone https://github.com/ebarron03/Project-2.git
```